Manual del instructor: Implementa IA en tus sistemas

Versión simplificada para alumnos de nuevo ingreso · Tec de Nogales · Angel Jesús Domínguez Contreras

Este manual acompaña la presentación (50 diapositivas), la carpeta de los alumnos (`cafe-nogal`) y tu carpeta (`cafe-nogal-instructor`). Cada diapositiva trae notas con lo que hay que decir. Regla de oro: **máximo 10 minutos hablando seguidos y luego que los alumnos tecleen algo.**

Mapa del taller (2 sesiones de 2 horas, con 15 min de descanso)

Sesión 1 · Las bases	Diapositivas	Min
Presentación y app terminada	1–4	5
Python desde cero (13 temas)	5–18	35
Descargar la carpeta + ejercicios en clase	19–21	20
Descanso	**22**	**15**
APIs e IA: qué son y qué puedes lograr	23–31	25
La app de Café Nogal y `chatbot.py` vacío	32–34	15
Llave de Gemini y cierre	35	5

Sesión 2 · El chatbot	Diapositivas	Min
Portada y repaso de ejercicios	36–37	5
Paso 1 · La llave	38	10
Paso 2 · Primera respuesta + cómo llega a la página	39–40	15
Paso 3 · Conocer el negocio	41–42	20
Descanso	**43**	**15**
Paso 4 · Memoria	44	15
Paso 5 · Seguridad y plan B	45–46	15
Paso 6 · Rompe el bot (pruebas y reto)	47	15
Bonus (consultar pedidos) y cierre	48–50	10

Si vas atrasado: recorta ejemplos de Python (chuleta, tipos de datos) o haz el bonus como demostración tuya. Nunca recortes los pasos 1 a 5 del chatbot.

Antes del taller (hoy, antes de empezar)

- En tu laptop: Python 3.12 o más nuevo y VS Code. Descomprime `cafe-nogal.zip` y corre la app una vez.
- Copia `sesion2_pasos/chatbot_paso6_bonus.py` sobre un `chatbot.py` de prueba con tu `.env` y corre `probar_chatbot.py`: así confirmas que tu llave y el modelo `gemini-3.5-flash` responden hoy.
- Abre en el navegador de la escuela `https://api.frankfurter.dev/v1/latest?base=USD&symbols=MXN`. Si la red lo bloquea, la app funciona igual; solo no muestra el dólar.
- Sube `cafe-nogal.zip` a donde lo vayan a descargar (Drive, Classroom o GitHub). **Nunca subas un `.env`.**
- Confirma con el laboratorio que `pip install` funciona (la red debe permitir pypi.org).

Sesión 1: Python, APIs y la app

Presentación (5 min · diap. 1–4)

Preséntate en un minuto y muestra la app terminada. *"Al final de la sesión 2 ustedes tienen esto funcionando y entienden cada línea."*

Python desde cero (35 min · diap. 5–18)

Ritmo: explicas 2 minutos, ellos escriben y corren 2 minutos. Mientras no tengan la carpeta, que creen un archivo `practica.py` en el escritorio; después usarán el de la carpeta.

- **¿Qué es Python? / ¿Qué puedes lograr?** Un programa es una receta que la computadora sigue al pie de la letra. Pregunta cuál de los 6 usos les llama más.
- **Primer programa:** `print` y cómo correr un archivo. Si `python` no se reconoce, usar `py`.
- **Variables y tipos:** el `=` significa "guarda en la caja". Insiste en que `"5"` no es `5`.
- **input y f-strings:** `input()` siempre regresa texto; para hacer cuentas, `int()`.
- **if, listas, diccionarios y for:** la sangría de 4 espacios es parte del código. Los diccionarios son lo que más usa la app.
- **Funciones (2 diapositivas):** es el tema más importante. `def` escribe la receta; llamarla la cocina. Diferencia `print` vs `return`: sale en los ejercicios. El chatbot es la función `responder(mensaje)`.
- **Leer errores:** lee un error real de alguien en voz alta, de abajo hacia arriba. Celebra los errores.

Descarga y ejercicios (20 min · diap. 19–21)

Todos descargan `cafe-nogal`, la abren en VS Code y corren los 3 comandos (crear `venv`, activarlo, `pip install`). Mientras se instala, explica los ejercicios y resuelve en vivo uno de cada módulo (diap. 21). Luego 15 minutos de trabajo con `python revisar.py`; quien termine ayuda a su compañero. Lo que no terminen, en casa.

Módulo 1 · Completa el código	Módulo 2 · Corrige el error
`total_pedido` (multiplicar), `saludo` (f-string), `tamano_bebida` (if/elif), `agregar_al_carrito` (append), `sumar_precios` (for), `precio_de` (diccionario)	10 % escrito como 10, `=+` en vez de `+=`, índice fuera de rango, texto por número, `print` en vez de `return`, `>` en vez de `>=`

APIs (25 min · diap. 23–31)

- **¿Qué es una API?** Analogía del mesero: tú pides, la cocina prepara, el mesero trae. No necesitas saber cómo está hecha la cocina.
- **¿Qué puedes lograr?** Pagos, mapas, mensajes, datos en vivo, iniciar sesión e IA. "No reinventas: conectas".
- **El dólar de hoy:** abre la URL en vivo y muestra el JSON; es igual a un diccionario.
- **Petición y respuesta:** 200 bien, 4xx culpa nuestra, 5xx culpa del servidor.
- **La llave:** nunca en el código; va en `.env`, que está en `.gitignore`.
- **API de IA e IA en tus sistemas:** modelo, tokens y que puede inventar. Recorre los 6 usos (chatbots, capturar información, reportes automáticos, clasificar, buscar en documentos, resumir). Pregunta cuál le harían al negocio de su familia.
- **Cómo funciona un chatbot:** el diagrama de 4 cuadros es el mapa de la sesión 2.

La app (15 min · diap. 32–35)

`streamlit run app.py`, recorren las páginas, leen juntos el fragmento de `app.py` y abren `chatbot.py`: solo tiene `return None`. Cierra pidiendo que creen su llave en `aistudio.google.com/app/apikey` antes de la sesión 2.

Sesión 2: construimos el chatbot

Todo se escribe en `chatbot.py`. Después de cada paso: guardar, `Ctrl + C`, `streamlit run app.py` y probar en la página Asistente. Si alguien se atrasa, copia el archivo de ese paso de `sesion2_pasos/` sobre su `chatbot.py`.

Repaso (5 min · diap. 37)

¿Cuántos llegaron a 12 de 12? Resuelve en voz alta `esta_abierto` (`> 7` → `>= 7`) y comparte las soluciones.

Paso 1 · La llave (10 min · diap. 38 · `chatbot_paso1.py`)

Copian `.env.example` como `.env`, pegan su llave y corren `python chatbot.py`: debe decir "¡Llave encontrada!". Si no: el archivo se llama `.env.txt` (Windows oculta extensiones), está en otra carpeta o hay espacios junto al `=`.

Paso 2 · Primera respuesta (15 min · diap. 39–40 · `chatbot_paso2.py`)

Agregan `from google import genai`, el cliente y `generate_content`. Momento "wow": la app ya responde. Luego preguntan el precio de un capuchino: inventa o no sabe. **Esa es la lección.** Con la diapositiva 40 explica que `app.py` solo llama a `responder()`; por eso el mismo bot podría ir en WhatsApp.

Paso 3 · Conocer el negocio (20 min · diap. 41–42 · `chatbot_paso3.py`)

La instrucción de sistema es el manual del empleado nuevo: quién es, qué sabe y qué reglas sigue. Leemos `negocio.json` y `productos.csv` tal cual con `leer_archivo` y los metemos en un f-string de triple comilla. Prueba: capuchino ($50) y té chai helado (no disponible). Demostración: cambia un precio en `productos.csv`, reinicia la app y el asistente ya lo sabe.

Paso 4 · Memoria (15 min · diap. 44 · `chatbot_paso4.py`)

Antes: "me llamo Ana" y luego "¿cómo me llamo?": no sabe. La API no recuerda nada; le reenviamos los últimos 20 mensajes con `armar_platica`. Aquí usan todo lo de la sesión 1: lista, for, if y diccionarios.

Paso 5 · Seguridad y plan B (15 min · diap. 45–46 · `chatbot_paso5.py`)

Primero pregunta "¿cómo romperían este bot?" y anota sus ideas; luego la tabla. Código: reglas de seguridad al final de las instrucciones, revisar el mensaje antes de gastar (vacío y máximo 500 letras) y el plan B con `try / except`: si el modelo principal falla (por ejemplo 503), prueba `gemini-3.5-flash-lite`. El archivo también avisa si falta la llave en lugar de tronar.

Paso 6 · Rompe el bot (15 min · diap. 47)

Copia `probar_chatbot.py` a la carpeta de cada alumno (junto a `chatbot.py`) y que revisen las 9 respuestas. Luego el reto por equipos: que invente un producto, que dé descuento, que revele sus instrucciones o que hable

de otra cosa. Quien lo logre lo comparte y entre todos escriben la regla que lo corrige.

Bonus y cierre (10 min · diap. 48–50 · `chatbot_paso6_bonus.py`)

Si hay tiempo, que lo escriban; si no, hazlo tú como demostración. La IA no lee el archivo de pedidos: pide que ejecutemos `consultar_pedido` y le damos el resultado (sin el nombre del cliente). Es el patrón para conectar la IA con cualquier sistema. Comparte la versión final y deja el reto: cambiar `datos/` por otro negocio. *"Usar la IA está bien; entender lo que hace es lo que te vuelve ingeniero."*

Cómo responder preguntas

- **¿Cuesta dinero?** El nivel gratuito de Gemini alcanza para aprender. En un negocio real se paga por tokens; por eso los límites.
- **¿Por qué Python?** Es fácil de leer y las bibliotecas de IA salen primero en Python. Lo que aprendieron sirve en cualquier lenguaje.
- **¿La IA puede mentir?** Sí, si no le das datos. Por eso las instrucciones con datos reales y las pruebas.
- **¿Puedo ponerlo en WhatsApp o en mi página?** Sí: `responder()` es la misma; solo cambia quién la llama.
- **Si no sabes algo:** "No lo sé, lo investigo y les digo". Anótalo.

Problemas en vivo

Síntoma	Causa probable	Solución
`python` no se reconoce	No se marcó "Add to PATH"	Usar `py` o reinstalar marcando la casilla
`venv\Scripts\activate` da error de permisos	PowerShell bloquea scripts	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` o usar la terminal "Command Prompt"
`ModuleNotFoundError`	No se instalaron los paquetes o el venv no está activo	Activar el venv y `pip install -r requirements.txt`
No aparece el dólar	La red bloquea la API	Ignorarlo: la app sigue funcionando
"No encontré la llave"	`.env` mal nombrado o en otra carpeta	Debe llamarse exactamente `.env` y estar junto a `chatbot.py`
Error 400 o 403	Llave mal copiada o cuenta sin nivel gratuito	Volver a copiarla o crearla con una cuenta personal de Google
Error 404 del modelo	Nombre de modelo mal escrito o retirado	Revisar `MODELO` en el `.env` (por ejemplo `gemini-3.6-flash`)
Error 429	Muchas peticiones seguidas	Esperar un minuto
Error 503	Modelo saturado	Esperar; desde el paso 5 el plan B lo resuelve solo
La app no cambia	No se reinició	`Ctrl + C` y `streamlit run app.py`